

Neste texto, Bertrand Meyer propõe uma abordagem metodológica focada na construção de softwares confiáveis, ou seja, corretos e robustos. O autor introduz uma teoria de design de software baseada na metáfora de que a construção de um sistema é uma sucessão de decisões de "contratação" documentadas entre componentes. A discussão utiliza a linguagem orientada a objetos Eiffel (criada pelo próprio autor) para demonstrar a implementação prática dessas ideias.

1. A Metáfora do Contrato e as Asserções

Na vida real, um contrato estabelece obrigações e benefícios mútuos para ambas as partes (cliente e contratado) de forma explícita.

No software, a relação entre uma rotina que chama (cliente) e a rotina que é chamada (fornecedor/contratado) deve operar da mesma forma.

Essa formalização é feita por meio de asserções:

Pré-condições (require): Representam a obrigação do cliente (o que deve ser verdadeiro antes da chamada) e o benefício do fornecedor (que não precisa se preocupar com estados inválidos).

Pós-condições (ensure): Representam a obrigação do fornecedor (o que ele garante entregar no final) e o benefício do cliente.

A regra de "sem cláusulas ocultas" estipula que, se a pré-condição não for atendida pelo cliente, a rotina não tem a obrigação de realizar absolutamente nada.

2. A Rejeição da "Programação Defensiva"

Meyer argumenta fortemente contra a prática comum da programação defensiva, que recomenda que todo módulo faça verificações redundantes para se proteger de todos os erros possíveis.

Segundo o autor, checagens cegas aumentam a complexidade do código e, paradoxalmente, diminuem a confiabilidade.

A teoria dos contratos prega que a responsabilidade deve ser única: se a condição está na pré-condição, é o cliente quem deve checar; se não está, o fornecedor deve lidar com a situação.

O autor defende o uso de "funções parciais", aceitando rotinas que possuem pré-condições rígidas e fazem apenas uma tarefa bem definida, ao invés de rotinas que tentam lidar com todos os inputs imagináveis.

3. Invariantes de Classe

Um invariante de classe é uma propriedade global que deve ser satisfeita por todas as instâncias de uma classe.

Ele atua como uma cláusula geral de contrato, devendo ser garantido logo após a criação do objeto e preservado antes e depois da execução de qualquer rotina exportada para os clientes.

4. Princípios do Tratamento de Exceções

Meyer define falha como a incapacidade de uma rotina cumprir o seu contrato.

Uma exceção ocorre quando uma sub-tarefa (ação) dentro de uma rotina falha.

O autor propõe três Leis do Tratamento de Exceções:

1. Uma rotina tem apenas duas formas de terminar: ou ela cumpre seu contrato, ou ela falha.

2. A falha de uma rotina deve sempre causar uma exceção na rotina que a chamou. (Ele critica linguagens como Ada que permitem que uma rotina falhe silenciosamente sem avisar o chamador).

3. Diante de uma exceção, uma rotina só pode reagir de duas formas: retomar (resumption - restaurar o estado e tentar novamente) ou entrar em "pânico organizado" (organized panic - restaurar o invariante da classe e reportar a falha ao chamador através de uma exceção).

Para implementar isso, a linguagem Eiffel oferece as palavras-chave rescue (para tratar a exceção e restaurar o estado limpo) e retry (para tentar executar o corpo da rotina novamente).

5. Herança e Vinculação Dinâmica (Subcontratação)

O autor esclarece o papel da herança aliada à redefinição de métodos, comparando-a a uma subcontratação.

Para que a vinculação dinâmica (dynamic binding) seja segura, um "subcontratado" (a classe herdeira que redefine um método) deve estar vinculado ao mesmo contrato da classe original.

Regra da Redefinição: Ao redefinir uma rotina, a nova pré-condição deve ser mais fraca (ou igual) à original, e a nova pós-condição deve ser mais forte (ou igual) à original. Isso significa que a nova versão pode aceitar mais casos e devolver resultados mais específicos, mas nunca exigir mais ou entregar menos do que a classe original prometeu.

Meyer também critica duramente a vinculação estática (static binding) em cenários polimórficos, considerando-a um grande erro, pois aplica a versão errada da rotina, podendo violar o invariante da subclasse.

Análise Conclusiva

O texto de Bertrand Meyer é um marco na engenharia de software ao trazer rigor matemático (oriundo de métodos formais e provas de corretude) para o design prático no dia a dia da orientação a objetos. Ao invés de tratar os erros como eventos caóticos que devem ser interceptados por blocos de código defensivos em todos os lugares, o Design by Contract transforma o desenvolvimento de software em um mapeamento claro de responsabilidades. Isso resulta em sistemas com arquiteturas mais simples, limpas e inerentemente mais fáceis de debugar e manter.